实验 8-存储器与控制器实验

一、 实验准备

本次实验开始涉及 MIPS 架构 CPU 的设计，其中涵盖 CPU 在流水线设计中所分割的两个阶段，以下为实验概述：MIPS 架构 CPU 的传统流程可分为取指、译码、执行、访存、回写(Instruction Fetch, Decode, Execution, Memory Request, Write Back) 五阶段。完成了执行阶段的 ALU 部分，并进行了简单的访存实验，本实验将实现取指、译码两个阶段的功能。在进行本次实验前，你需要具备以下实验环境及基础能力：

1. 了解 Xilinx Block Memory Generator IP 的使用
2. 了解数据通路、控制器的概念

二、 实验目的

2.1 了解随机存取存储器 RAM 的原理。
2.2 掌握调用 Xilinx 库 IP(Block Memory Generator)实例化 RAM 的方法；
2.3 掌握单周期 CPU 各个控制信号的作用和生成过程。
2.4 掌握单周期 CPU 控制器的工作原理及其设计方法。
2.5 理解单周期 CPU 执行指令的过程。
2.6 掌握取指、译码阶段数据通路、控制器的执行过程。

三、 实验设备

PC 机一台，Basys3 开发板，Xilinx Vivado 开发套件。

四、 实验任务

4.1 实验要求

图 1 为本次实验所需完成内容的原理图，依据取指、译码阶段的需求，分别需要实现以下模块:

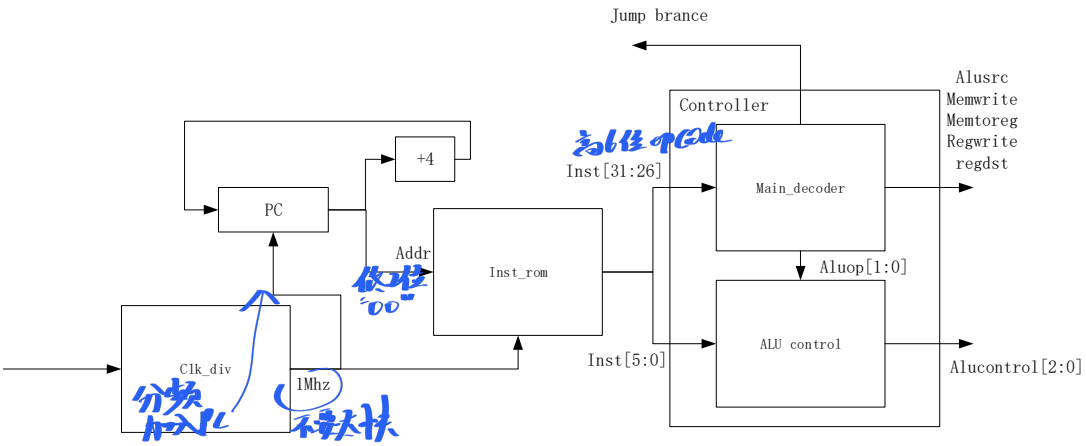


图 1

1. PC。 D 触发器结构，用于储存 PC(一个周期)。需实现 2 个输入，分别为 clk, rst, 分别连接时钟和复位信号；需实现 2 个输出，分别为 pc, inst_ce，分别连接指令存储器的 addra, ena 端口。其中 addra 位数依据 coe 文件中指令数定义；
2. 加法器。用于计算下一条指令地址，需实现 2 个输入，1 个输出，输入值分别为当前指令地址 PC、32'h4；

3. Controller。其中包含两部分：
(a). main_decoder。负责判断指令类型，并生成相应的控制信号。需实现 1 个输入，为指令 inst 的高 6 位 op，输出分为 2 部分，控制信号有多个，可作为多个输出，也作为一个多位输出，具体参照 4.3 进行设计；aluop 传输至 alu_decoder，使 alu_decoder 配合 inst 低 6 位 funct，进行 ALU 模块控制信号的译码。
(b). alu_decoder。负责 ALU 模块控制信号的译码。需实现 2 个输入，1 个输出，输入分别为 funct, aluop；输出位 alucontrol 信号。
(c). 除上述两个组件，需设计 controller 模块顶层文件调用两个 decoder，对应实现 op,funct 输入信号，并传入调用模块；对应实现控制信号及 alucontrol，并连接至调用模块相应端口。
4. 指令存储器。使用 Block Memory Generator IP 构造。 (参考存储器实验)
5. 时钟分频器。将板载 50Mhz 频率降低为 1Mhz，连接 PC、指令存储器时钟信号 clk。

为了便于同学们查看板上信号是否正常，我们提供了一个串口监视器，可用于监视控制信号的状态，同时可以通过串口发送指令来控制 CPU 的状态。同学们只需要在我们提供的模板上实现代码，并将对应的信号线连接上即可对 CPU 进行监视与控制。

4.2 实验步骤

1. 创建工程
2. 导入提供的 proc_top, main_dec, alu_dec, controller 模板，导入 sys_monitor, uart_top, uart_rx, uart_tx 模块，根据实验原理中的说明例化需要使用的 IP 核
3. 创建 PC，clk_div 模块，编写其中的逻辑
4. 编写 main_dec, alu_dec, controller 中的逻辑（逻辑参照表 2 和表 4）
5. 使用 Block Memory，导入 coe 文件（根据存储器实验）
6. 将 proc_top 定义为顶层模块，连接串口监视器，烧录程序查看实验效果

4.3 实验原理

4.4.1 取指阶段原理

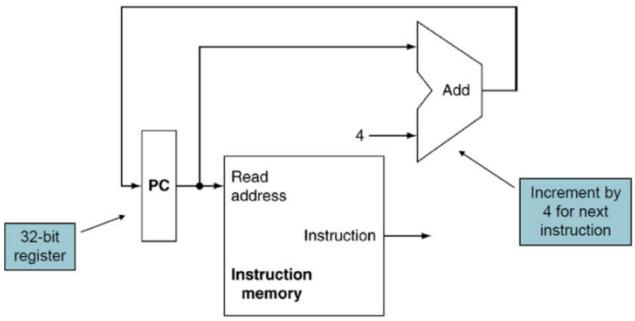


图 2

如图 2 所示，PC 为 32bit(1 word)的寄存器，其存放指令地址，每条指令执行完毕后增加 4，即为下一条指令存放地址。指令地址传入指令存储器，即可取出相应地址存放的指令。需要注意的是，MIPS 架构中，采用字节读写，32bit word = 4 byte，故需要地址 +4 来获取下一条指令。

4.4.2 指令译码原理

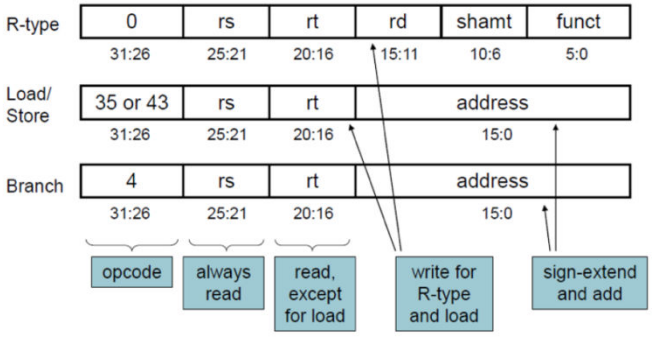


图 3

如图 3 所 32 位 MIPS 指令在不同类型指令中分别有不同结构。但[31:16]表示的 opcode，以及 [5:0] 表示的 funct，为译码阶段明确指令控制信号的主要字段。表 3 为 opcode 及 funct 识别得到的部分信号，详细信号表课堂 PPT。

表 2

opcode	aluop	operation	funct	alu function	alu control
lw	00	Load word	XXXXXX	Add	010
sw	00	Store word	XXXXXX	Add	010
beq	01	Branch equal	XXXXXX	Subtact	110
R-type	10	Add	100000	qdd	010
		subtract	100010	Subtract	110
		and	100100	And	000
		or	100101	Or	001
		set-on-less-than	101010	SLT	111

4.4.3 控制器实现原理

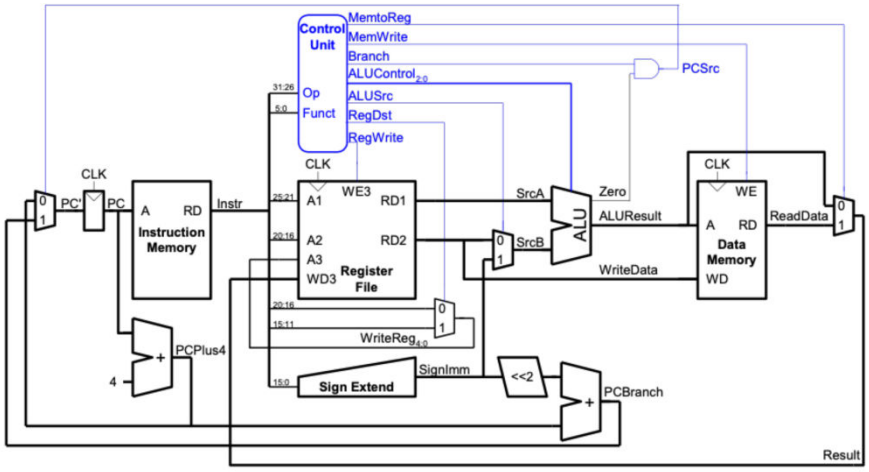


图 4

由图 4 可知，控制器输出的控制信号，用于控制器件的使能和多路选择器的选择，因此，根据不同指令的功能分析其所需要的路径，即可得到信号所对应的值。在此之前，参照下对各个控制信号的含义进行理解。

表 3

信号	含义
memtoreg	回写的数据来自于 ALU 计算的结果/存储器读取的数据
memwrite	是否需要写数据存储器
pcsrc	下一个 PC 值是 PC+4/跳转的新地址
alusrc	送入 ALU B 端口的值是立即数的 32 位扩展/寄存器堆读取的值
regdst	写入寄存器堆的地址是 rt 还是 rd,0 为 rt,1 为 rd
regwrite	是否需要写寄存器堆
branch	是否为 branch 指令,且满足 branch 的条件
jump	是否为 jump 指令
alucontrol	ALU 控制信号,代表不同的运算类型

分析数据通路图，判断指令是否需要写寄存器、访存等等操作，以产生相应的控制信号。下面给出参考信号表：

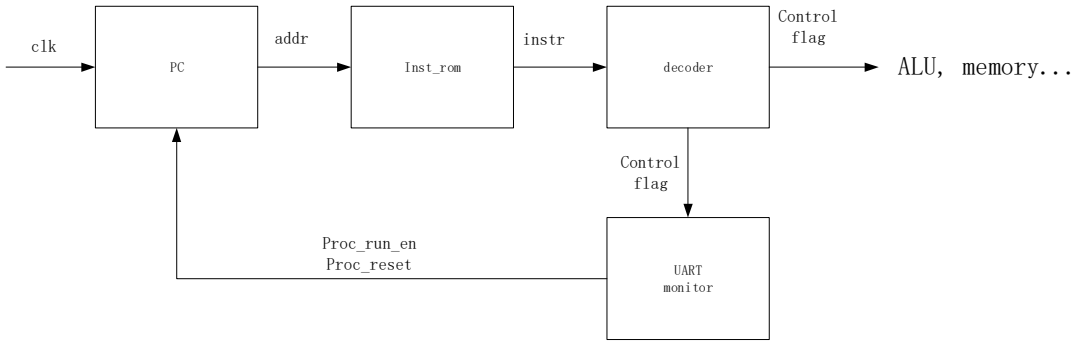
表 4

instruction	op5:0	regwrite	regdst	alusrc	branch	memWrite	memtoReg	aluop1:0
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	X	1	0	1	X	00
beq	000100	0	X	0	1	0	X	01
addi	001000	1	0	1	0	0	0	00
j	000010	0	X	X	X	0	X	XX

表四里面没有定于 pcsrc，这里 pcsrc 只用于决定 beq 的 PC 选择，因此可以定义为：pcsrc = branch & zero。

4.4 串口监视器说明

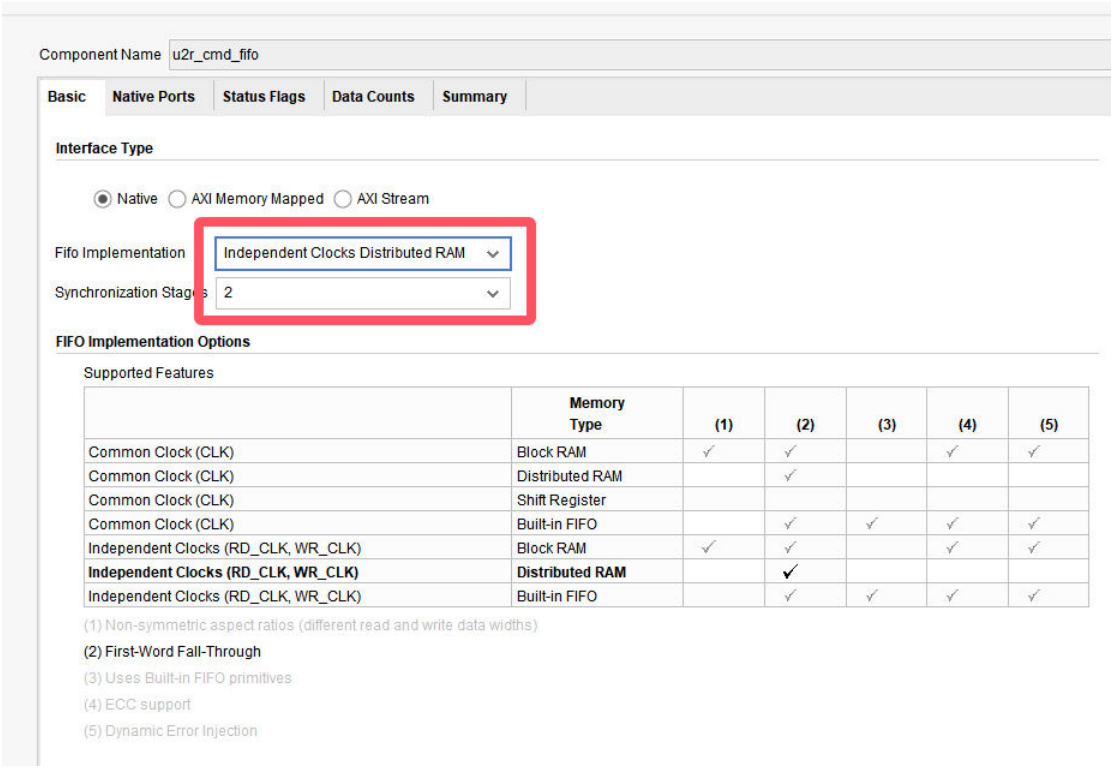
首先对串口监视器的功能进行说明。串口监视器在整体 CPU 中所处的位置如下图所示(有简化)：

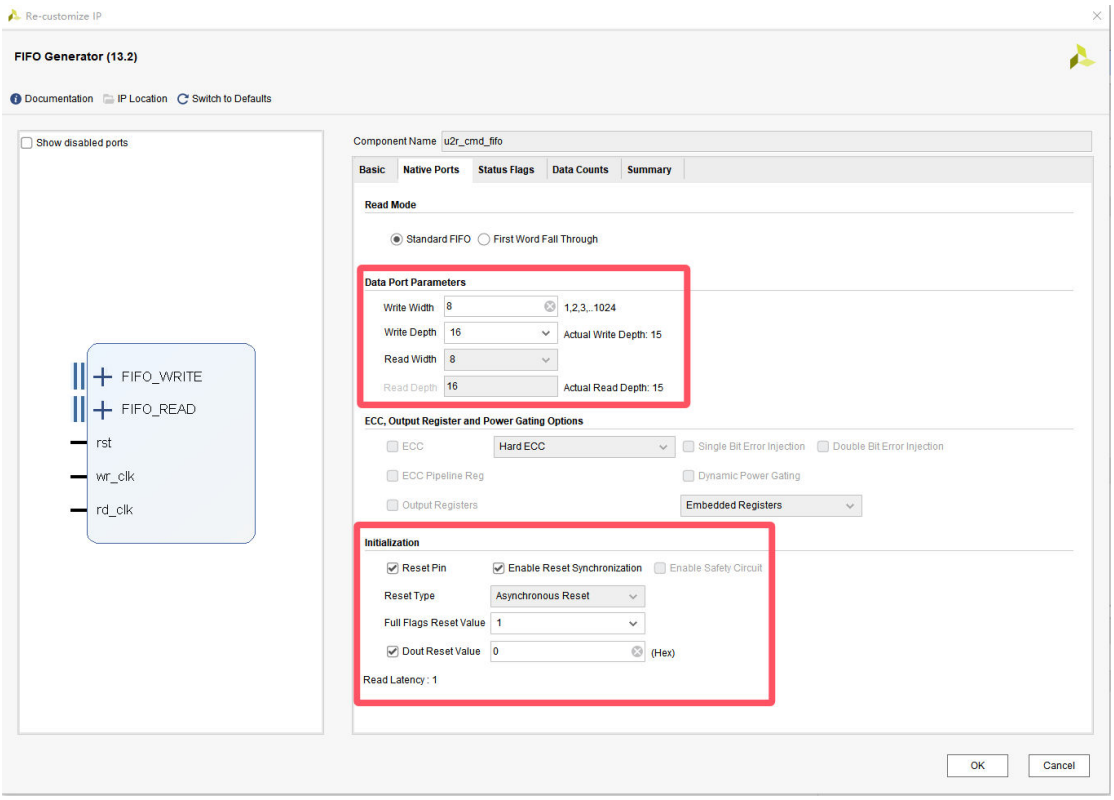


本次实验需要查看的控制信号会在之后的实验中对 ALU、寄存器组等组件进行操作，但本次实验中，我们先不管他，直接接到监视器上通过串口进行打印。监视器同时还负责对 PC 进行控制，用于控制 CPU 的启动与软重置。

为了使用该监视器，我们还需要例化如下两个 FIFO：u2r_cmd_fifo、r2u_data_fifo，具体配置如下：

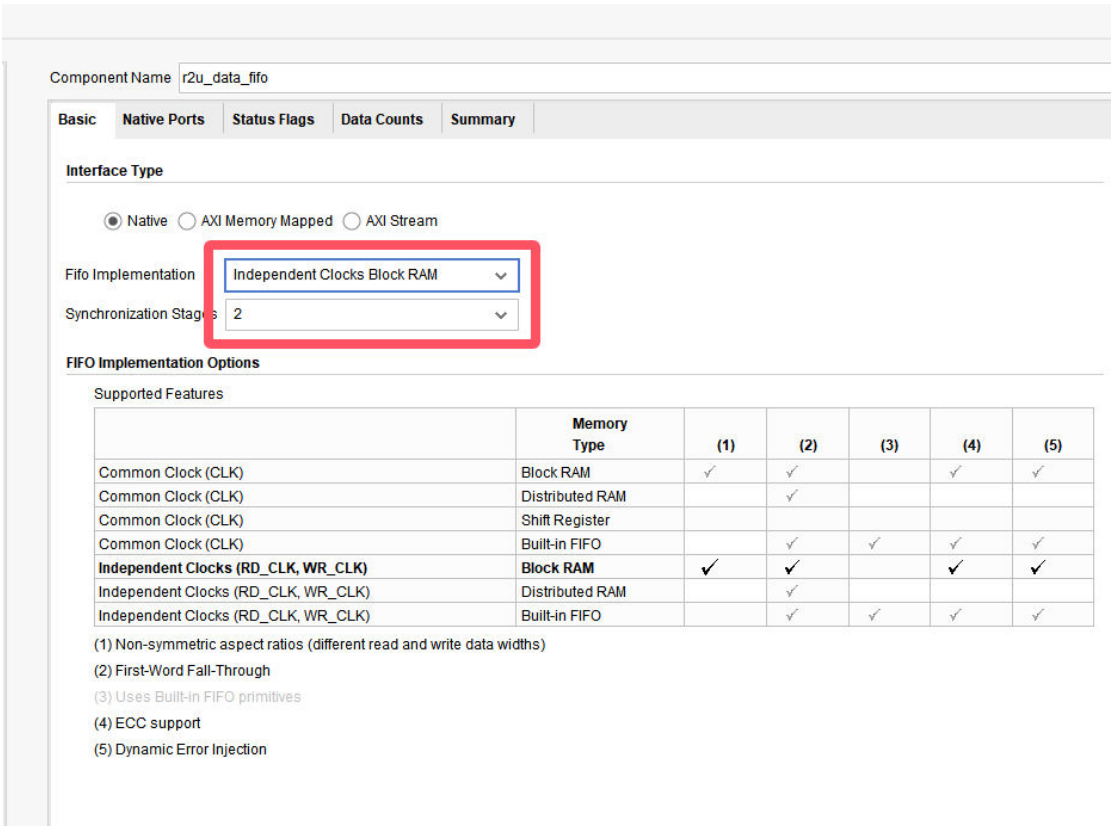
u2r_cmd_fifo

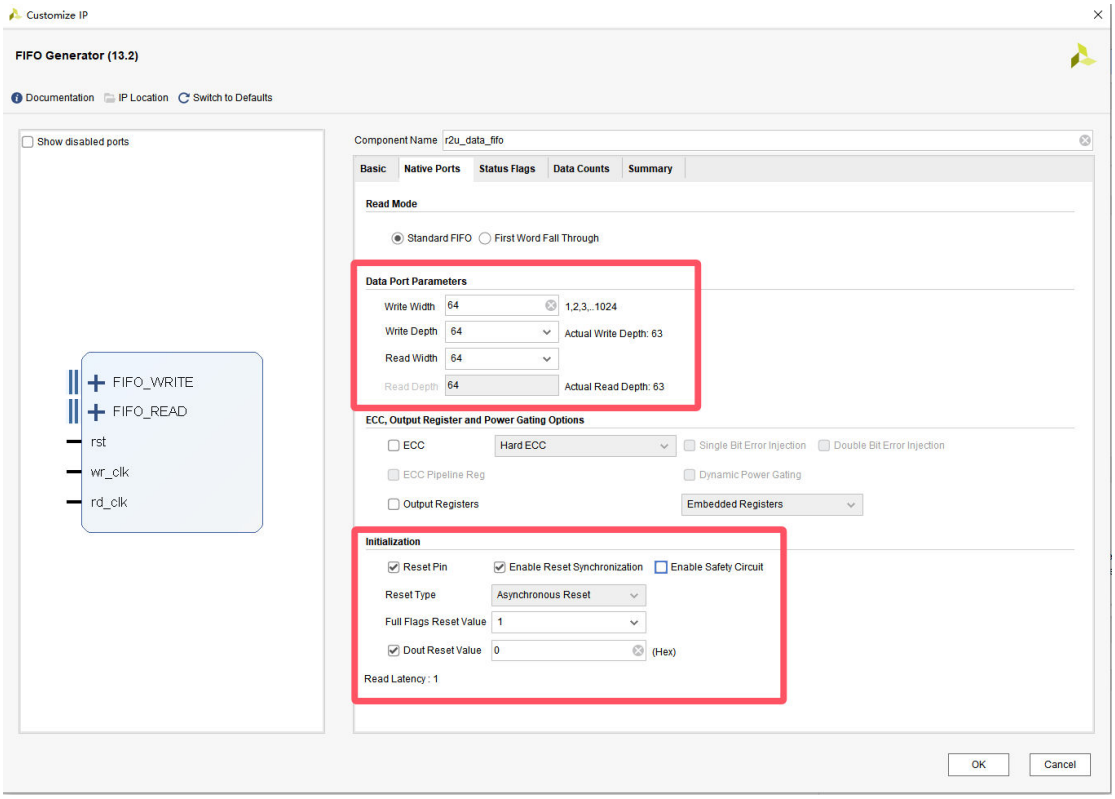




一定要注意 Initialization 中的 Enable Reset Synchronization 和 Enable Safely Circuit。如果没有勾选 Enable Reset Synchronization，FIFO 的复位端会变为 wr_rst 与 rd_rst 两个端口，都需要连接复位信号。如果勾选了 Enable Safely Circuit，FIFO 会多出 wr_rst_busy 和 rd_rst_busy 用于指示 FIFO 的复位状态。

r2u_data_fifo:

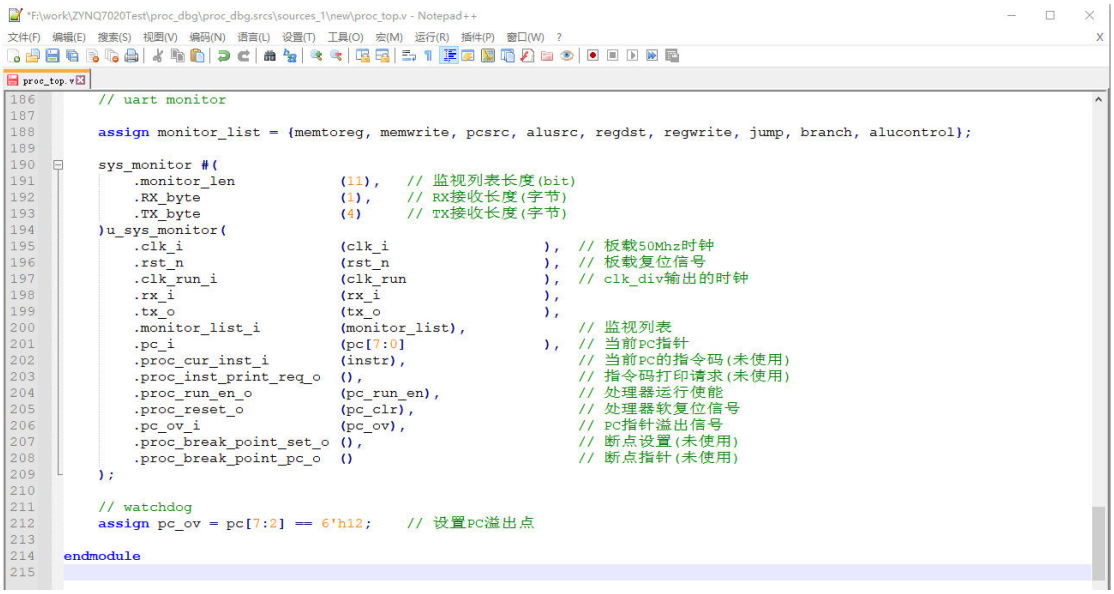




同样需要注意红框内的勾选状态。

这两个 FIFO 的目的是完成处理器与串口之间的数据交互。由于我们的处理器频率与串口的工作时钟不一致，需要通过 FIFO 完成数据传输，否则会造成很多不正常的响应。

接下来介绍串口监视器的顶层接口，点开 proc_top.v，声明如下：

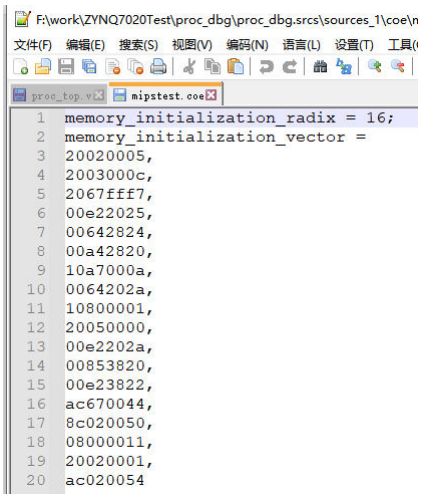


```
186     // uart monitor
187
188     assign monitor_list = {memtoreg, memwrite, pcsrc, alusrc, regdst, regwrite, jump, branch, alucontrol};
189
190     sys_monitor #(
191         .monitor_len            (11),    // 监视列表长度(bit)
192         .RX_byte                (1),     // RX接收长度(字节)
193         .TX_byte                (4)      // TX接收长度(字节)
194     )u_sys_monitor(
195         .clk_i                  (clk_i                  ),  // 板载50Mhz时钟
196         .rst_n                  (rst_n                  ),  // 板载复位信号
197         .clk_run_i              (clk_run                ),  // clk_div输出的时钟
198         .rx_i                   (rx_i                   ),
199         .tx_o                   (tx_o                   ),
200         .monitor_list_i         (monitor_list),             // 监视列表
201         .pc_i                   (pc[7:0]                ),  // 当前PC指针
202         .proc_cur_inst_i        (instr),                    // 当前PC的指令码(未使用)
203         .proc_inst_print_req_o  (),                         // 指令码打印请求(未使用)
204         .proc_run_en_o          (pc_run_en),                // 处理器运行使能
205         .proc_reset_o           (pc_clr),                   // 处理器软复位信号
206         .pc_ov_i                (pc_ov),                    // PC指针溢出信号
207         .proc_break_point_set_o (),                         // 断点设置(未使用)
208         .proc_break_point_pc_o  ()                          // 断点指针(未使用)
209     );
210
211     // watchdog
212     assign pc_ov = pc[7:2] == 6'h12;    // 设置PC溢出点
213
214 endmodule
215
```

其中标识了未使用的信号为之后的实验使用的端口，本次实验不需要连接。
你需要做的有以下三点：

1. 将自己的 controller 中输出的控制信号接到 monitor_list 上，并计算总长度填入 monitor_len 中；
2. 将自己的 clk_div 中分出的 1Mhz 时钟接到 clk_run 上，将自己的 PC 和从 inst_rom 中读出的 instr 接到对应端口上；
3. 使用 pc_run_en 和 pc_clr 控制你的 PC 计数器。pc_run_en 在串口上接到启动指令后会一直拉高，直到 pc_ov 拉高时 pc_run_en 拉低避免 PC 跑飞；pc_clr 在在串口上接到复位指令后会拉高一周期，此时 PC 应重置为 0；此时 PC 应该就是一个**带使能与复位端的计数器**(不会写的可以自行百度这个名词)；当 pc_run_en 为高电平时，PC 会在每个周期自增 4，直至 pc_run_en 拉低；当 pc_clr 为高电平时，PC 置为 0；
4. 根据你向 inst_rom 中写入的指令长度，更改 pc_ov 的判断条件。

指令长度的计算样例如下：



```
1  memory_initialization_radix = 16;
2  memory_initialization_vector =
3  20020005,
4  2003000c,
5  2067fff7,
6  00e22025,
7  00642824,
8  00a42820,
9  10a7000a,
10 0064202a,
11 10800001,
12 20050000,
13 00e2202a,
14 00853820,
15 00e23822,
16 ac670044,
17 8c020050,
18 08000011,
19 20020001,
20 ac020054
```

这个 coe 中一共有 18 条指令，因此 PC 能指向的 inst_rom 最大地址应该就是 18，也就是十六进制的 12，因此将 pc_ov 设置为 6'h12。

串口的控制指令有两条：

指令名	指令码	说明
启动	00	启动 CPU，开始执行指令
复位	01	复位 PC 指针

为了便于调试，当处理器开始执行程序后，串口监视器会在每一条指令执行时打印当前的 PC 以及 monitor_list 中的内容，便于同学们检查。

如果你没有动之前的 monitor_list，现在串口接收到的数据格式比特位如下：

alusrc	regdst	regwrite	jump	branch	alucontrol	memtoreg	memwrite	pcsrc	pc
0	1	2	3	4	[5:7]	8	9	10	[24:31]

其中[11:23]被填充为 0。

在使用串口监视器控制 CPU 运行后，请将 pc_run_en 接到指令 ROM 的使能端，避免在 PC 未开始运行时，指令 ROM 频繁取出第一条指令运行，干扰处理器状态。

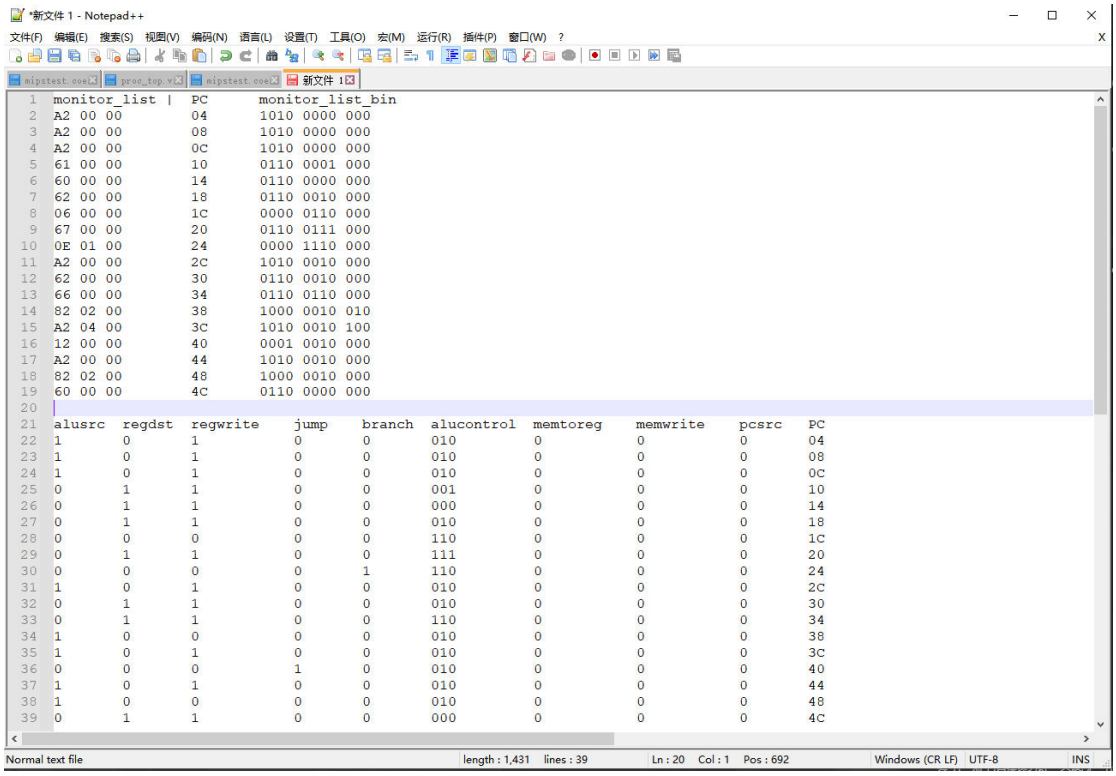
```
84      inst_rom u_inst_rom (
85          .clka(clk_run),     // input wire clka
86          .ena(pc_run_en),       // input wire ena
```

4.5 实验操作与现象

生成比特流后，打开串口调试助手，发送 00，查看回传的数据：



串口助手中的数据已经被包装为 16 进制，将它复制下来，随便找个编辑器打开复制进去，自行整理一下：

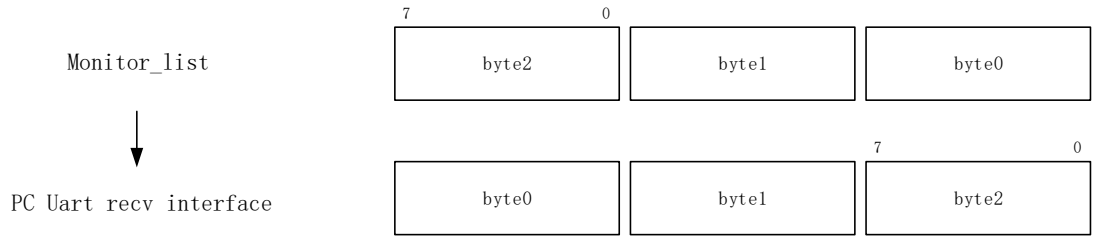


```
monitor_list |  PC      monitor_list_bin
A2 00 00        04      1010 0000 000
A2 00 00        08      1010 0000 000
A2 00 00        0C      1010 0000 000
61 00 00        10      0110 0001 000
60 00 00        14      0110 0000 000
62 00 00        18      0110 0010 000
06 00 00        1C      0000 0110 000
67 00 00        20      0110 0111 000
0E 01 00        24      0000 1110 000
A2 00 00        2C      1010 0010 000
62 00 00        30      0110 0010 000
66 00 00        34      0110 0110 000
82 02 00        38      1000 0010 010
A2 04 00        3C      1010 0010 100
12 00 00        40      0001 0010 000
A2 00 00        44      1010 0010 000
82 02 00        48      1000 0010 000
60 00 00        4C      0110 0000 000

alusrc  regdst  regwrite  jump  branch  alucontrol  memtoreg  memwrite  pcsrc  PC
1       0       1         0     0       010         0         0         0      04
1       0       1         0     0       010         0         0         0      08
1       0       1         0     0       010         0         0         0      0C
0       1       1         0     0       001         0         0         0      10
0       1       1         0     0       000         0         0         0      14
0       1       1         0     0       010         0         0         0      18
0       0       0         0     0       110         0         0         0      1C
0       1       1         0     0       111         0         0         0      20
0       0       0         0     1       110         0         0         0      24
1       0       1         0     0       010         0         0         0      2C
0       1       1         0     0       010         0         0         0      30
0       1       1         0     0       110         0         0         0      34
1       0       0         0     0       010         0         0         0      38
1       0       1         0     0       010         0         0         0      3C
0       0       0         1     0       010         0         0         0      40
1       0       1         0     0       010         0         0         0      44
1       0       0         0     0       010         0         0         0      48
0       1       1         0     0       000         0         0         0      4C
```

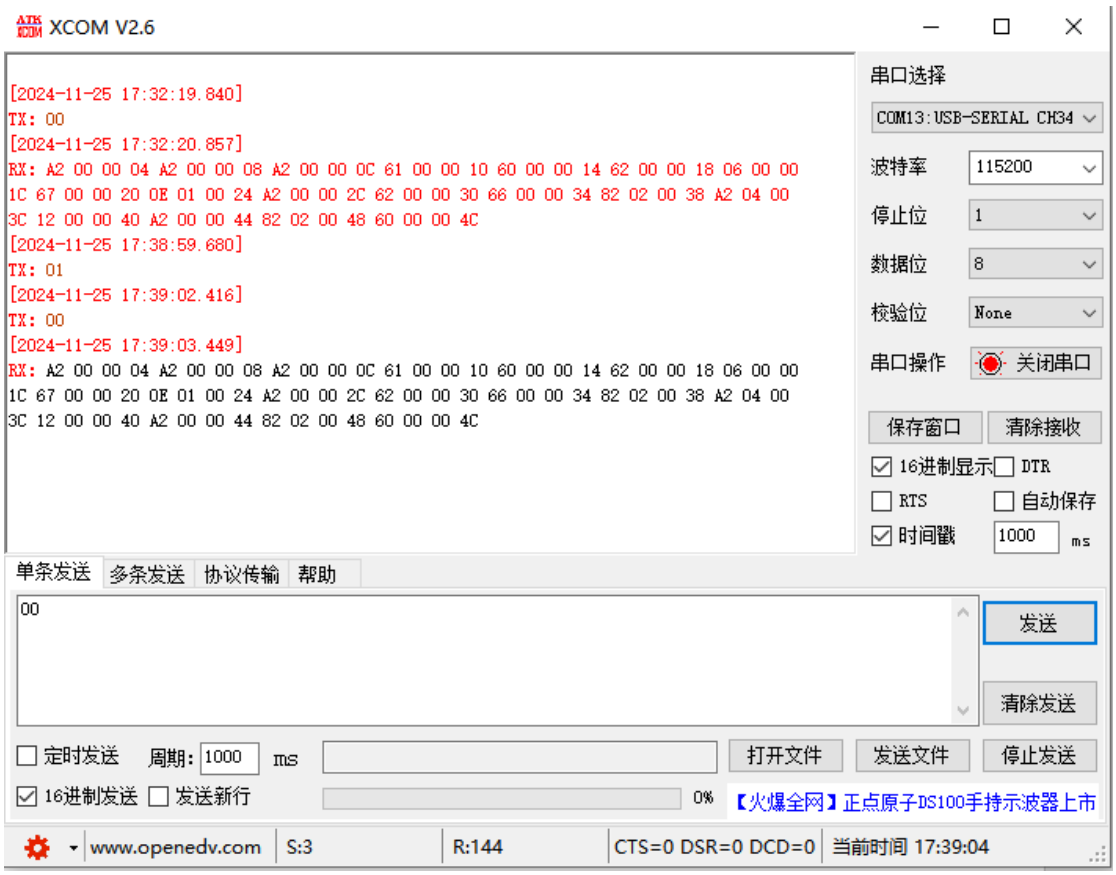
需要注意，本处演示的 CPU 已经能处理指令，且载入的 coe 包含一个完整的运算程序，可能与同学们的效果不一致。同学们可以自己编写指令观察控制器的效果是否符合预期。

由于传输时数据存在大小端的问题，传到串口上的数据顺序可能与 monitor_list 中的顺序不一致，以下是一个数据顺序转换的样例：



如果觉得自己整理这个数据太麻烦，可以写个快速转换的程序。

如果需要重新测试数据，可以通过串口调试助手发送 01 后再次发送 00：



4.6 实验报告要求

1. 按照实验要求 4.1 搭建如图 1 所示实验系统，需要在存储器实验的基础上，需要自己实现实现分频 clk_div.v, D 触发器结构 pc.v，控制器 controller.v 模块，其中 controller.v 包含 main controller 和 ALU controller。

2. 在指令存储器里面依次放入 R-type (例如 add 和 sub)，lw, sw, beq，ori 指令的 32 位机器码（表 4）。根据读出的指令类型，通过串口去观察控制器的输出结果。

3. 在实验报告中，阐述实验设计过程，代码模块结构，以及观察到实验结果，并分析实验结构。